

ESG Buddy — Complete Technical Overview

1. Root Program — What Starts the App

The entry point is `start_esgbuddy.bat` at the project root. It opens two terminal windows:

- **Backend:** `cd backend → activate venv → python -m uvicorn app.main:app --reload --port 8000`
- **Frontend:** `cd frontend → npm run dev (port 3000)`

The root file that boots the backend is `backend/app/main.py`, and the root file that boots the frontend is `frontend/src/main.jsx`.

2. What Each File Does

2.1 Backend — `backend/app/`

main.py — The root FastAPI application. Defines ALL API routes (upload, evaluate, reports, clauses, accuracy, system). On startup, it loads documents metadata and compliance reports from JSON, parses ESG standards into clauses (or loads from ChromaDB), and initializes all global instances (vector store, compliance pipeline, accuracy evaluator, ground truth loader, clause parser).

config.py — Centralized settings. Uses Pydantic BaseSettings to read the `.env` file. Controls: OpenAI API key, LLM model, embedding model, chunk size/overlap, top-k retrieval, whether reflection is enabled, parallel batch size, GRI scope, which frameworks to parse, and directories.

models.py — All data models (Pydantic). Defines `ComplianceStatus`, `ESGFramework`, `EvidenceType`, `ESGClause`, `DocumentChunk`, `RetrievedEvidence`, `ValidationRule`, `RuleValidationResult`, `LLMEvaluation`, `ClauseEvaluation`, `ComplianceReport`, `GroundTruthLabel`, `AccuracyMetrics`, and more.

ingestion.py — PDF processing. `DocumentProcessor` extracts text from PDFs using PyMuPDF page by page, cleans text, splits into overlapping token-based chunks (default 512 tokens, 50 overlap using tiktoken), and generates document IDs. `EmbeddingGenerator` calls OpenAI's text-embedding-3-small to create 1536-dimensional vectors.

clause_parser_enhanced.py — Parses ESG standard PDFs into structured `ESGClause` objects. Has framework-specific regex parsers for BRSR (core metrics plus Section A/B/C) and GRI (disclosure extraction). Supports configurable GRI scope filtering (core/standard/essential). Can also use LLM-based parsing for SASB.

vector_store.py — ChromaDB interface. Manages two collections: `company_documents` (uploaded doc chunks) and `esg_clauses` (parsed standard clauses). Provides `add_document_chunks()`, `search_documents()` (semantic search with L2 to similarity conversion), `add_clauses()`, `get_all_clauses()`, `clear_clauses()`, and `delete_document()`.

compliance_pipeline.py — The core AI evaluation engine. For each clause: (1) semantic retrieval, (2) LLM evaluation with framework-specific prompts, (3) rule validation, (4) final decision combining LLM

plus rules. Contains the agentic pipeline (Chain-of-Thought, Reflection, Revision). Processes clauses in parallel batches.

rule_validator.py — Deterministic rule-based validation — the manual rule checking engine. Four rule types: numeric, temporal, keyword, and field_presence.

ground_truth_loader.py — Loads expert-labelled ground truth from JSON files. Maps company names (TCS, RIL, TATA Motors) to their ground truth files. Converts compliance labels to system status. Filters by system clause IDs.

accuracy.py — Accuracy measurement. Computes Precision, Recall, F1 by comparing system predictions to ground truth. Also computes retrieval recall at k, rule override precision, confidence calibration error, and self-benchmark stats.

clause_parser.py — Original (simpler) clause parser, superseded by clause_parser_enhanced.py.

2.2 Frontend — frontend/src/

main.jsx — React entry point. Renders the App component into the DOM.

App.jsx — Router setup with React Router. Defines routes: / (Home), /upload, /documents, /reports, /reports/:id (ReportDetail), and /clauses.

components/Layout.jsx — Shared layout with navigation sidebar/header.

pages/Home.jsx — Dashboard with system stats (documents, reports, clauses).

pages/Upload.jsx — PDF upload with drag-and-drop and progress.

pages/Documents.jsx — Lists uploaded documents, triggers compliance evaluation.

pages/Reports.jsx — Lists all compliance reports with summary cards and compliance rates.

pages/ReportDetail.jsx — The most complex page. Shows summary stats, accuracy metrics, human verification dashboard (approve/reject ambiguous clauses with expandable details), clause list with status filters, expandable clause cards showing AI analysis, evidence, and rule results.

pages/Clauses.jsx — Browse parsed ESG clauses by framework.

3. Manual Rule Checking — The Rules We Have

The rule engine lives in backend/app/rule_validator.py (RuleValidator class). It runs four types of deterministic checks on the retrieved evidence text.

3.1 Numeric Rule (_validate_numeric)

Extracts all numbers from evidence text using regex. Checks if any number falls within a configured [min_value, max_value] range. Example: A clause requiring GHG emissions data — the rule checks that a numeric value exists in the evidence.

3.2 Temporal Rule (_validate_temporal)

Has three sub-modes:

- **Year mode:** Finds 4-digit years (19xx or 20xx) and checks they fall in [min_year, max_year]

- **Date mode:** Finds dates in MM/DD/YYYY, MM-DD-YYYY, or YYYY-MM-DD formats
- **Period mode:** Looks for keywords like "period", "fiscal year", "quarter", "reporting period"

Example: A clause requiring data for a specific reporting period — the rule checks that a valid year or date reference exists.

3.3 Keyword Rule (`_validate_keyword`)

Takes a list of required keywords. Can operate in `match_all` mode (ALL keywords must appear) or `match_any` mode (at least one). Performs case-insensitive matching against the evidence text. Example: A clause about water management — the rule checks for keywords like "water", "consumption", "recycled".

3.4 Field Presence Rule (`_validate_field_presence`)

Checks whether specific field labels appear in the evidence followed by a colon or equals sign. Uses regex pattern matching. All specified fields must be present for the rule to pass. Example: A clause requiring "Scope 1" and "Scope 2" disclosures — the rule checks that both field labels appear as data fields.

3.5 How Rules Interact with the LLM

In `compliance_pipeline.py`, the `_make_final_decision()` method combines LLM output with rule results:

- If a **mandatory rule fails** and the LLM said "supported" or "partial", the system **overrides to partial** and caps confidence at 0.5.
- If **all rules pass** but the LLM said "not_supported", the system lowers the LLM's confidence by 0.2 (rules suggest compliance but LLM disagrees — trust LLM but with less certainty).
- The final confidence is always averaged with the rule pass rate: $\text{final_confidence} = (\text{llm_confidence} + \text{rule_pass_rate}) / 2$.

4. System Prompts — Where They Are and What They Say

All system prompts are in `backend/app/compliance_pipeline.py`. They are passed as the "system" role message in OpenAI API calls. There are six system prompts across four methods:

4.1 Fast Evaluation — BRSR system prompt (line 297)

"You are a BRSR disclosure compliance expert. BRSR is about DISCLOSURE PRESENCE, not fact verification. Focus on whether the required information is disclosed, not whether it's sufficient or accurate."

4.2 Fast Evaluation — GRI system prompt (line 300)

"You are an ESG Compliance Analyzer for GRI. Prefer Supported when evidence substantively addresses the clause (narrative, policy, table, cross-ref). Use Inferred only when evidence is clearly indirect. Minimize Partial and Not Supported."

4.3 Fast Evaluation — Default system prompt (line 303)

"You are an ESG compliance analyst. Be concise and objective."

4.4 Chain-of-Thought — BRSR system prompt (line 430)

"You are a BRSR disclosure compliance expert. BRSR is about DISCLOSURE PRESENCE. Use supported, partial, inferred, or not_supported as appropriate."

4.5 Self-Reflection system prompt (line 559)

"You are a critical reviewer who identifies flaws and inconsistencies in ESG compliance analysis."

4.6 Revision system prompt (line 627)

"You are an expert ESG analyst who revises analysis based on critical feedback."

Additionally, there are three framework-specific user prompts (the long detailed instructions sent as the "user" role message):

- `_get_brsr_prompt()` (line 317) — BRSR disclosure presence evaluation with four classification labels (Supported, Partial, Inferred, Not Supported) and specific rules about cross-references, NA/Nil handling, and preference ordering.
- `_get_gri_prompt()` (line 381) — GRI substantive evidence assessment that prefers Supported when evidence clearly addresses the requirement, reserves Inferred for genuinely indirect evidence, and uses Partial rarely.
- `_get_default_prompt()` (line 353) — Generic ESG evaluation for TCFD and SASB.

5. Agentic Components

The agentic AI pipeline is in `backend/app/compliance_pipeline.py`, controlled by the `enable_reflection` setting in `config.py` (currently set to `False` for speed, can be toggled to `True`).

When enabled, each clause goes through a three-step agentic reasoning loop:

5.1 Step 1: Chain-of-Thought Reasoning (`_chain_of_thought_reasoning`, line 414)

The LLM receives the clause, evidence, and structured reasoning steps. For BRSR, the steps are: check disclosure presence, cross-references, explicit NA/Nil, partial assessment, inferred assessment, and not supported determination. For GRI, the steps are: does evidence substantively address the clause, inferred only when indirect, partial rarely, and not supported only when blank. The output includes `reasoning_steps` (a list), `status`, `confidence`, `explanation`, and `detailed_reasoning`.

5.2 Step 2: Self-Reflection (`_self_reflection`, line 505)

A second LLM call acts as a critical reviewer of the Step 1 output. It checks six dimensions: logical consistency, evidence coverage, bias, completeness, alternative interpretations, and confidence calibration. The output includes a reflection summary, a list of issues, a list of strengths, a `needs_revision` boolean, and `revision_suggestions`.

5.3 Step 3: Revision (`_revise_reasoning`, line 572)

This step only runs if `needs_revision` is `True` from Step 2. A third LLM call receives the original analysis plus the issues identified by the reviewer. It produces a corrected analysis that addresses the identified problems. The output includes a revised `status`, `confidence`, `explanation`, `detailed_reasoning`, and a `changes_made` description.

When disabled (current default), clauses go through `_fast_evaluation()` — a single LLM call with framework-specific prompts, no reflection.

The `LLMEvaluation` model in `models.py` stores the full agentic trace: status, confidence, explanation, reasoning, `reasoning_steps` (chain-of-thought), reflection (self-reflection text), `reflection_issues` (issues found), and `revised` (boolean indicating whether revision occurred).

6. How the Whole App Works End-to-End

Step 1: Startup

`main.py` `startup_event` loads saved documents metadata and reports from JSON files. For each enabled framework (BRSR, GRI, TCFD, SASB), it either loads clauses from ChromaDB or parses them from standard PDFs and indexes them into the vector store.

Step 2: User uploads a PDF

`POST /documents/upload` triggers `ingestion.py` which extracts text page by page with `PyMuPDF`, chunks it (512 tokens, 50 overlap), generates embeddings via `OpenAI text-embedding-3-small`, and stores chunks plus embeddings in ChromaDB.

Step 3: User triggers evaluation

`POST /compliance/evaluate` with document ID and framework. For each clause in that framework, the pipeline runs in parallel batches of 10:

- **Retrieval:** Searches ChromaDB for top-8 relevant chunks using the clause as a search query
- **LLM Evaluation:** Sends evidence plus clause to `GPT-4o-mini` with framework-specific prompt (optionally with agentic Chain-of-Thought, Reflection, and Revision)
- **Rule Validation:** Runs deterministic numeric/temporal/keyword/field checks
- **Final Decision:** Combines LLM output with rule results, applies overrides if mandatory rules fail

Step 4: Report is generated and persisted

The compliance report is saved to `data/compliance_reports.json` with all evaluations, summary statistics, and metadata.

Step 5: User views report

The frontend fetches the summary and clause evaluations. It shows the compliance rate, status distribution (supported, partial, inferred, not supported), and expandable clause cards with AI analysis, evidence, and rule results.

Step 6: Human verification

Ambiguous clauses (partial, inferred, or confidence below 70%) are shown in the Human Verification dashboard with Approve and Reject buttons. `POST /compliance/override` updates the evaluation and recomputes the report summary in real time.

Step 7: Accuracy measurement

GET /accuracy/metrics/{report_id} triggers the ground truth loader to find the matching company file. The accuracy evaluator computes Precision, Recall, and F1 by comparing system predictions against expert labels.